

STACK OPERATIONS AND AMORTIZED COST ANALYSIS

Aim

To implement stack operations using a dynamically resizing array and calculate the amortized cost of push operations.

Algorithm

1. Initialize the stack array, top pointer, cost and push counter.
2. Read the number of push operations from the user.
3. For each push operation, resize the array when full and insert the element.
4. Display the pushed element and the current top position of the stack.
5. Read the number of pop operations, remove elements and display the updated top position.
6. Calculate and display the total cost, number of pushes and amortized cost.

Program

```
import java.util.Scanner;

public class StackAmortized {
    static int[] a = new int[2];
    static int top = -1;
    static long cost = 0, pushes = 0;

    static void push(int x) {
        if (top + 1 == a.length) {
            int old = a.length, n = old * 2;
            int[] b = new int[n];
            for (int i = 0; i <= top; i++) b[i] = a[i];
            a = b; cost += old;
        }
        a[++top] = x; cost++; pushes++;
    }

    static int pop() { return a[top--]; }
    static double amortized() { return pushes == 0 ? 0 : (double) cost / pushes; }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter no. of pushes: ");
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            push(i);
            System.out.println("Pushed: " + i + " -> stack top = " + top);
        }
        System.out.println("Enter no. of pops: ");
        int p = sc.nextInt();
        for (int i = 0; i < p && top >= 0; i++) {
            int x = pop();
            System.out.println("Popped: " + x + " -> stack top = " + top);
        }
        System.out.println("Total cost = " + cost);
        System.out.println("Pushes = " + pushes);
        System.out.printf("Amortized cost = %.2f\n", amortized());
    }
}
```

Output

```
Enter no. of pushes:
Pushed: 1 -> stack top = 0
Pushed: 2 -> stack top = 1
```

```
Pushed: 3 -> stack top = 2
Pushed: 4 -> stack top = 3
Pushed: 5 -> stack top = 4
Enter no. of pops:
Popped: 5 -> stack top = 3
Popped: 4 -> stack top = 2
Total cost = 11
Pushes = 5
Amortized cost = 2.20
```

Result

Thus, the program to implement stack operations and calculate the amortized cost has been executed successfully. The stack push and pop operations were performed correctly and the amortized cost was calculated successfully.